S.no	**Topic**	Subtopic (Project work included)
2	**CSS**	**1) CSS Introduction** **2) CSS Syntax** **3) CSS Selectors** **4) CSS How To** **5) CSS Comments** **6) CSS Colors** **7) CSS Backgrounds** **8) CSS Borders** **9) CSS Margins** **10) CSS Padding** **11) CSS Height/Width** **12) CSS Box Model** **13) CSS Outline** **14) CSS Text** **15) CSS Fonts** **16) CSS Icons** **17) CSS Links** **18) CSS Lists** **19) CSS Tables** **20) CSS Display** **21) CSS Max-width** **22) CSS Position** **23) CSS Z-index** **24) CSS Overflow** **25) CSS Float** **26) CSS Inline-block**

		27) CSS Combinators **28) CSS Dropdown** **29) CSS Pseudo-classes** **30) CSS Pseudo-element** **31) CSS RWD** **32) CSS RWD Images** **33) CSS RWD Videos** **34) CSS Grid** **35) CSS Navigation Bar** **36) CSS Rounded Corners** **37) CSS Border Images** **38) CSS Multiple Backgrounds** **39) CSS Color & Color Keywords** **40) CSS Gradients** **41) CSS Shadows** **42) CSS Text-effects** **43) CSS 2D Transforms** **44) CSS 3D Transforms** **45) CSS Transitions** **46) CSS Animations** **47) CSS Tooltips** **48) CSS Style Images** **49) CSS Image Reflection**

1. CSS Introduction

CSS (Cascading Style Sheets) is a language used to describe the style of an HTML document. It controls how HTML elements are displayed on screen, paper, or in other media. **CSS can be applied in three ways: Inline, Internal, and External.**

Use Case Scenario:

Suppose you're designing a blog page and you want all your headings to be blue and the body text to be in Arial. Instead of applying styles to each tag individually, CSS allows you to style multiple elements at once efficiently.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
font-family: Arial;
background-color: #f0f0f0;
}
h1 {
color: blue;
}
</style>
</head>
<body>
<h1>Welcome to My Blog</h1>
<p>This is my first styled webpage using CSS.</p>
</body>
</html>
```

This is **Internal css**. It can be **applied using <style>** tag in head section. & we can use different kind of selectors. Here we have used Element **selector "Body"**

2. CSS Syntax

A CSS rule-set consists of a selector and a declaration block:

selector {

property: value;

}

Note:- The selector points to the HTML element, and the declaration block contains one or more declarations separated by semicolons.

Use Case Scenario:

You want all paragraphs in your site to have justified text alignment and a font size of 16px.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
text-align: justify;
font-size: 16px;
}
</style>
</head>
<body>
<p>This paragraph is justified and has a font size of 16px.</p>
</body>
</html>
```

3. CSS Selectors

Selectors are used to target the HTML elements you want to style. Common selectors include:

1. **Universal selector:- ***
2. **Element selector:- p, h1, etc.**
3. **ID selector:- # id**
4. **Class selector:- . class**
5. **Grouping selector:- h1, p**

Use Case: You want to style all buttons with a class of .btn to have a green background.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.btn {
background-color: green;
color: white;
padding: 10px 20px;
border: none;
border-radius: 5px;
}
</style>
</head>
<body>
<button class="btn">Click Me</button>
</body>
</html>
```

Here we have used **"class"** selector named as **.btn** in <style> section & in body section we have linked in **<button class="btn">**

5.CSS Comments:-

CSS comments can be added **as // :- single line**

For multiline:-

/* statement 1

Statement-2*/

6. CSS Colors

CSS supports colors using:

1. **Name: red, blue:-**

   Ex:- color: red,blue;

2. **HEX: #ff0000**

   Ex:- color: #ff0000

3. **RGB: rgb(255, 0, 0)**

   Ex:- color: rgb(255,0,0)

4. **HSL: hsl(0, 100%, 50%)**

   Ex:- color:hsl(0,100%,50%)

Use Case: You want to give a soft background color to a section of your site.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
section {
background-color: #f9f9f9;
color: #333;
padding: 20px;
}
</style>
</head>
<body>
<section>
<h2>This is a section with custom background and text color</h2>
<p>The color values are applied using HEX and named color.</p>
</section>
</body>
</html>
```

7. CSS Backgrounds

CSS background properties allow you to set background color, images, positioning, repeat options, and more.

Use Case:

You want to set a full-page background image.

Here are all background properties with all methods.

7.1 Background-image:-

Sets one or more background images for an element.

background-image: none | url("path") | gradient;

Value	Description
none	No background image (default).
url("image.jpg")	Specifies the path to the background image.
linear-gradient(...)	Applies a linear gradient.
radial-gradient(...)	Applies a radial gradient.
repeating-linear-gradient(...)	Repeating linear gradient background.
repeating-radial-gradient(...)	Repeating radial gradient background.
multiple	Allows multiple background images separated by commas.

Example 1:- background-image: url("image.jpg");

Example 2:- background-image: linear-gradient(to right, red, yellow);

Example 3:- background-image: url("img1.jpg"), url("img2.png");

7.2 background-repeat

Defines how a background image will repeat.

background-repeat: repeat | no-repeat | repeat-x | repeat-y | space | round;

Value	Description
repeat	Repeats both horizontally and vertically. *(default)*
no-repeat	Shows the image only once.
repeat-x	Repeats only horizontally.
repeat-y	Repeats only vertically.
space	Evenly spaces the image without cropping.
round	Stretches/repeats the image to fill space exactly.

Example 1:- background-repeat: no-repeat;

Example 2:- background-repeat: repeat-x;

Example 3:- background-repeat: space;

7.3 background-size:- **specifies the size of the background image.**

background-size: auto | cover | contain | [width] [height];

Value	Description
auto	Original size of the image.
cover	Covers the entire area (may crop).
contain	Fits entire image in the container (no crop).
length	Specific size like 100px 50px.

Value	Description
percentage	Relative to the element’s size like 100% 100%.

Example 1:- **background-size: cover;**

Example 2:-background-size: contain;

Example 3:- **background-size: 200px 100px;**

Example 4:- background-size: 50% 50%;

7.4 background-position

Specifies the position of the background image.

```
background-position: keyword | x% y% | xpx ypx;
```

Value	Description
left, center, right	Horizontal positions.
top, center, bottom	Vertical positions.
Combination	E.g. top left, center center, right bottom
Length/Percentage	E.g. 20px 10px, 50% 50%

Example 1: background-position: center;

Example 2:- background-position: top left;

Example 3:- background-position: 100px 50px;

7.5 background (Shorthand)

Combines all background-* properties into one line.

```
background: [color] [image] [position]/[size] [repeat] [attachment];
```

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
background-image: url('https://www.w3schools.com/w3images/lights.jpg');
background-repeat: no-repeat;
background-size: cover;
background-position: center;
}
</style>
</head>
<body>
<h1>Full Background Image Example</h1>
</body>
</html>
```

8. CSS Borders

Borders are used to define the outline around elements. CSS provides various border properties to style borders:

8.1 border-width: Sets the width of the border.
8.2 border-style: Sets the style (solid, dashed, dotted, etc.).
8.3 border-color: Sets the color of the border.
8.4 border-radius: Rounds the corners of the border.
8.5 border: A **shorthand property** for setting width, style, and color.

border-top, border-right, border-bottom, border-left: For individual border sides.

Use Case: You want to highlight a card element with different border styles for each side, rounded corners, and color variations.

Practical Example:

```
<!DOCTYPE html><html><head>
<style>
.border-box {
border-width: 4px;
border-style: dashed;
border-color: red;
padding: 15px;
margin-bottom: 20px;
}
.border-shorthand {
border: 3px solid green;
padding: 15px;
margin-bottom: 20px;
}
.border-radius {
border: 2px solid blue;
border-radius: 10px;
padding: 15px;
margin-bottom: 20px;
}
.border-sides {
border-top: 2px solid orange;
border-right: 4px dotted purple;
border-bottom: 2px dashed brown;
border-left: 4px double teal;
padding: 15px;
}
</style></head><body>
<div class="border-box">
This box uses separate border-width, border-style, and border-color.
</div>
<div class="border-shorthand">
This box uses the shorthand <code>border</code> property.
</div>
<div class="border-radius">
This box has rounded corners using <code>border-radius</code>.
</div>
<div class="border-sides">
This box has individual styles for each border side.
</div></body>
</html>
```

9. CSS Margin

The margin property sets the space outside the border of an element. It helps in spacing out elements from one another. You can set margins for individual sides using:

margin-top,margin-right,margin-bottom,margin-left

Or use the shorthand margin property:

margin: 20px (all sides)
margin: 20px 10px (top/bottom, left/right)
margin: 20px 15px 10px 5px (top, right, bottom, left)

Use Case: You want to control the spacing between cards or sections dynamically.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.box1 {
margin: 40px; /* all sides */
background-color: lightyellow;
}
.box2 {
margin: 30px 50px; /* top-bottom and left-right */
background-color: lightblue;
}
.box3 {
margin: 10px 20px 30px 40px; /* top, right, bottom, left */
background-color: lightgreen;
}
</style>
</head>
<body>
<div class="box1">Box 1 - Equal margin on all sides</div>
<div class="box2">Box 2 - Top/Bottom and Left/Right margin</div>
<div class="box3">Box 3 - Custom margin for each side</div>
</body>
</html>
```

10. CSS Padding

Padding is the space between the content of the element and its border. Similar to margin, it can be set using:

padding-top,padding-right,padding-bottom,padding-left

Or shorthand:

```
padding: 20px

padding: 20px 10px

padding: 20px 15px 10px 5px
```

Use Case : You want inner spacing for an element so content does not touch the borders.

Practical Example:

```html
<!DOCTYPE html>
<html>
<head>
<style>
.box1 {
padding: 20px;
background-color: lightcoral;
}
.box2 {
padding: 10px 40px;
background-color: lightseagreen;
}
.box3 {
padding: 5px 10px 15px 20px;
background-color: lightgray;
}
</style>
</head>
<body>
<div class="box1">Box 1 - Equal padding</div>
<div class="box2">Box 2 - Vertical and Horizontal padding</div>
<div class="box3">Box 3 - Individual padding values</div>
</body>
</html>
```

11. CSS Height / Width

The height and width properties in CSS are used to define the dimensions of HTML elements. These dimensions can be fixed, relative to the parent, or based on the viewport.

These properties help in setting up responsive layouts, controlling element sizing, and ensuring consistent design across devices.

Property	Values	Description
width	auto	Default value; the browser calculates the width.
	<length> (e.g., 400px, 50em)	Fixed width using units like px, em, rem.
	<percentage> (e.g., 80%)	Relative to the width of the parent container.
	initial	Sets the property to its default value.
	inherit	Inherits the width from its parent.
height	Same as above	Applies to height with same value types.
max-width	<length> / <percentage>	Restricts the maximum width an element can grow to.
min-width	<length> / <percentage>	Ensures a minimum width.
max-height	<length> / <percentage>	Restricts the maximum height an element can grow to.
min-height	<length> / <percentage>	Ensures a minimum height.

Use Case: You want all image containers to have the same fixed height but adjust width based on the screen.

Practical Example:

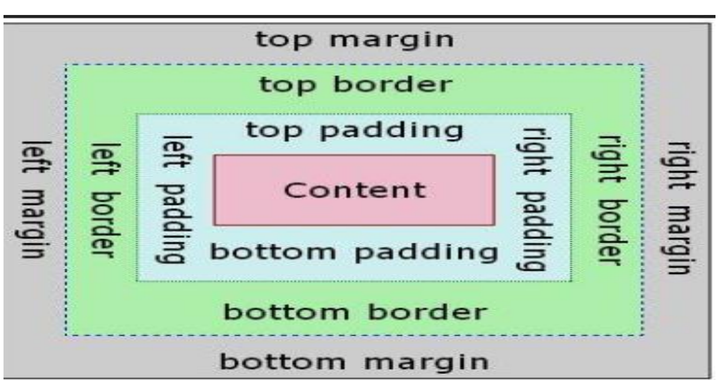
```
<!DOCTYPE html>
<html>
<head>
<style>
.box1 {
 height: 200px;
 width: 300px;
 background-color: lightblue;
}

.box2 {
 height: 100px;
 width: 50%;
 background-color: lightgreen;
}
</style>
</head>
<body>
<div class="box1">Fixed Size Box</div>
<div class="box2">Responsive Width Box</div>
</body></html>
```

12.CSS Box Model

The CSS box model describes the rectangular boxes that are generated for elements in the document tree. Every element is a **box composed of four areas:**

1. Content – The actual content of the box, like text or images.
2. Padding – Clears an area around the content. It is transparent.
3. Border – A border that goes around the padding and content.
4. Margin – Clears an area outside the border. It is also transparent.



Use Case: You want to design a card-like layout with some spacing inside and outside the box and a border for separation.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.box {
width: 300px;
padding: 20px;      /* Space inside the box */
border: 5px solid #000; /* Border around the padding */
margin: 30px;       /* Space outside the border */
background-color: #f1f1f1;
}
</style>
</head>
<body>
<div class="box">
This is a styled box using the box model.
</div></body></html>
```

This structure helps maintain spacing, alignment, and layout consistency throughout the webpage.

13. CSS Outline

The outline in CSS is a line drawn outside the element's border edge. It's commonly used to highlight or focus elements (especially in form fields or accessibility contexts). Unlike borders, outlines do **not take up space** and cannot have individual sides styled.

Use Case Scenario:

To highlight an input field on focus or a button when it's selected — without shifting the layout or taking up extra space like borders would.

Property	Values	Description
outline-style	solid, dotted, dashed, double, groove, ridge, inset, outset, none, hidden	Defines the appearance of the outline.
outline-	color name, hex, rgb(), hsl(), transparent	Sets the color of the outline.

Property	Values	Description
color		
outline-width	thin, medium, thick, length (e.g., 2px)	Specifies the thickness of the outline.
outline-offset	<length> (e.g., 5px)	Adds space between the outline and the border edge.
outline	width style color	Shorthand to set all three properties at once.

Note: outline is often used for accessibility: keyboard users rely on outlines to see where focus is. **Use outline: none;** cautiously — ensure you provide a custom focus style if removing the default.

Practical Example:-

```
<!DOCTYPE html><html>
<head>
<style>
.input-box {
  padding: 10px;
  border: 1px solid #ccc;
  outline: none;
}

.input-box:focus {
  outline-style: dashed;
  outline-color: #e17055;
  outline-width: 3px;
  outline-offset: 4px;
}
</style>
</head>
<body>
<input class="input-box" type="text" placeholder="Focus me to see outline">
</body>
</html>
```

14. CSS Text

CSS provides a variety of properties to control text formatting including color, text-align, text-decoration, text-transform, letter-spacing, and line-height.

Use Case: You want headings to be uppercase and centered.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
h2 {
text-align: center;
text-transform: uppercase;
color: navy;
}
</style>
</head>
<body>
<h2>Section Title</h2>
</body>
</html>
```

15. CSS Fonts

CSS allows you to control the font family, style, size, weight, and more to improve text readability and aesthetics.

Use Case: You want all the text on your site to use a modern, sans-serif font and ensure readability.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
body
{
```

```
font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
font-size: 18px;
}
</style>
</head>
<body>
<p>This paragraph is using a clean sans-serif font with 18px size.</p>
</body>
</html>
```

16. CSS Icons CSS allows icons to be used via:

Method	Syntax/Example	Description
Font Awesome	<i class="fa fa-home"></i>	Uses a library of scalable icons that can be styled with CSS.
Unicode	content: "\1F4E7"; inside ::before	Adds native emoji or symbol using content.
Background	background-image: url('icon.png');	Uses image as icon in a div or span.

Use Case: You want to show a home icon beside a “Home” button or link using scalable vector icons instead of images.

Practical Example: (Using Font Awesome)

```
<!DOCTYPE html><html><head>
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
<style>
.icon-button {
  font-size: 20px;
  padding: 10px;
}
</style></head>
<body>
<button class="icon-button"><i class="fa fa-home"></i> Home</button>
</body></html>
```

17. CSS Links

You can style links using their states: link, visited, hover, and active.

Pseudo-class	Description
:link	Unvisited link
:visited	Link the user has already visited
:hover	When the mouse pointer is over the link
:active	While the link is being clicked (active state)
:focus	When the link is selected or focused (e.g. by tabbing)

Important: Always follow this order when styling links:-

a:link → a:visited → a:hover → a:active

(otherwise styles might not be applied correctly due to specificity rules)

Use Case Scenario:

You want links to change color on hover and remove the underline.

Practical:-

```
<!DOCTYPE html><html>
<head>
<style>
a:link {
  color: blue;
  text-decoration: none;
}
a:visited {
  color: purple;
}
a:hover {
  color: red;
  text-decoration: underline;
}
a:active {
  color: orange;
  background-color: #f1f1f1;
```

```
}
a:focus {
  outline: 2px dashed #55efc4;
  padding: 4px;
}
</style></head>
<body>
<h2>CSS Link States Demo</h2>
<p>
  <a href="https://www.example.com" target="_blank">Visit Example.com</a>
</p>
<p>
  <a href="https://www.w3schools.com" target="_blank">Visit W3Schools</a>
</p>
</body></html>
```

18. CSS Lists

CSS can style both ordered (<ol>) and unordered (<ul>) lists, as well as list items (<li>).

Property	Values / Examples	Description
list-style-type	disc, circle, square, decimal, upper-roman, lower-alpha, etc.	Defines the type of bullet or numbering.
list-style-position	inside, outside	inside: bullets are inside content box; outside (default): to the left.
list-style-image	url("bullet.png")	Replaces default bullet with a custom image.
list-style	Shorthand for type position image	Combines all above into one declaration.

You can change bullet types, use custom images, control spacing, and adjust positions.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
ul.custom {
  list-style-type: square;
  list-style-position: inside;
  padding-left: 20px;
}

ol.roman {
  list-style-type: upper-roman;
}

ul.image-bullets {
  list-style-image: url('https://www.w3schools.com/css/sqpurple.gif');
}
</style>
</head>
<body>
<h3>Square Bullets</h3>
<ul class="custom">
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
<h3>Roman Numbering</h3>
<ol class="roman">
  <li>Install</li>
  <li>Setup</li>
  <li>Deploy</li>
</ol>
<h3>Custom Image Bullets</h3>
<ul class="image-bullets">
  <li>Item One</li>
  <li>Item Two</li>
</ul>
</body></html>
```

19. CSS Tables

CSS gives complete control over the look of HTML tables. You can style headers, rows, borders, spacing, alignment, and even add zebra-striping.

Property	**Values / Examples**	**Description**
border-collapse	collapse, separate	Collapse merges borders; separate keeps them distinct.
border-spacing	<length> (e.g. 10px)	Sets space between borders in separate mode.
text-align	left, center, right	Horizontal alignment of cell content.
vertical-align	top, middle, bottom	Vertical alignment of cell content.
padding	<length>	Controls spacing inside each cell.
nth-child(even/odd)	Selects alternating rows for styling	Common for zebra-striping effect.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
table {
  width: 80%;
  margin: auto;
  border-collapse: collapse;
}

th, td {
  padding: 12px;
  border: 1px solid #636e72;
  text-align: center;
}

th {
  background-color: #2d3436;
  color: white;
```

```
}

tr:nth-child(even) {
  background-color: #dfe6e9;
}
</style>
</head>
<body>
<h2 style="text-align:center;">Styled Table</h2>
<table>
  <tr>
    <th>Language</th>
    <th>Usage</th>
  </tr>
  <tr>
    <td>HTML</td>
    <td>Structure</td>
  </tr>
  <tr>
    <td>CSS</td>
    <td>Styling</td>
  </tr>
  <tr>
    <td>JavaScript</td>
    <td>Interactivity</td>
  </tr>
</table>
</body>
</html>
```

20. CSS Display

The display property specifies the display behavior (the type of rendering box) of an element. Common values:

20.1 Block: Element takes full width, starts on a new line.
20.2 Inline: Element does not start on a new line, only takes as much width as required.
20.3 Inline-block: Behaves like inline, but allows setting height/width.
20.4 None: Hides the element (it is not displayed at all).
20.5 Flex: Makes an element a flex container.
20.6 Grid: Makes an element a grid container.

Use Case Scenario:

You want to convert inline elements into blocks or use layout models like flex/grid.

Practical Example:

```
<!DOCTYPE html>

<html>
<head>
<style>
.inline-example {
display: inline;
background: pink;
}

.block-example {
display: block;
background: lightblue;
}

.inline-block-example {
display: inline-block;
width: 100px;
background: lightgreen;
}

.hidden-example {
display: none;
}
</style>
</head>
<body>
<span class="inline-example">Inline</span>
<span class="block-example">Block</span>
<span class="inline-block-example">Inline-Block</span>
<span class="hidden-example">You won't see me</span>
</body>
</html>
```

21. CSS Max-width

The max-width property defines the maximum width an element is allowed to grow to. It is useful in creating responsive designs, as it allows elements to scale but prevents them from stretching too far.

When width is set in percentage and max-width is set in pixels (or any fixed unit), it ensures that the element grows responsively up to a limit.

Property	Values / Examples	Description
max-width	none	Default. No maximum width limit.
	<length> (e.g., 600px, 40em)	Restricts width to a specific fixed value.
	<percentage> (e.g., 80%)	Sets max width relative to parent container.
	initial	Sets to browser's default.
	inherit	Inherits the max-width value from parent element.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.container {
  width: 90%;
  max-width: 600px;
  background-color: #dff9fb;
  margin: 20px auto;
  padding: 20px;
  font-family: Arial, sans-serif;
  box-shadow: 0 0 10px rgba(0,0,0,0.1);
}

img.responsive {
  width: 100%;
  max-width: 500px;
  height: auto;
  display: block;
```

```
 margin: auto;
}
</style>
</head>
<body>

<div class="container">
 <h2>Responsive Box with Max-width</h2>
 <p>This content box will grow up to a maximum of 600px, and then stop expanding
further even if the screen is wider. </p>
</div>

<img class="responsive" src="https://www.w3schools.com/w3images/lights.jpg"
alt="Sample Image" />

</body>
</html>
```

22. CSS Position

The position property specifies how an element is positioned in a document. The top, right, bottom, and left properties determine the final position:

- **22.1 Static:** Default value. Elements flow normally.
- **22.2 Relative:** Positioned relative to its normal position.
- **22.3 Absolute:** Positioned relative to the nearest positioned ancestor.
- **22.4 Fixed:** Positioned relative to the browser window.
- **22.5 Sticky:** Switches between relative and fixed depending on scroll.

Use Case Scenario:

You want to pin a header at the top, or place a tooltip over a button.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.static-box {
position: static;
background-color: lightgray;
```

```
}
.relative-box {
position: relative;
top: 10px;
left: 20px;
background-color: lightblue;
}
.absolute-container {
position: relative;
}
.absolute-box {
position: absolute;
top: 0;
right: 0;
background-color: lightgreen;
}

.fixed-box {
position: fixed;
bottom: 0;
right: 0;
background-color: coral;
padding: 10px;
}

.sticky-box {
position: sticky;
top: 0;
background-color: yellow;
}
</style>
</head>
<body>
<div class="static-box">Static Position</div>
<div class="relative-box">Relative Position</div>
<div class="absolute-container" style="height: 100px; border: 1px solid #000;">
<div class="absolute-box">Absolute Inside Relative Container</div>
</div>
<div class="sticky-box">Sticky Header - scroll down</div>
<div style="height: 1000px;"></div>
<div class="fixed-box">Fixed Box</div>
</body>
</html>
```

23. CSS Z-index

The z-index property in CSS controls the **vertical stacking** order of overlapping elements. **Higher z-index values appear in front of lower ones.** It only works on elements that have a position other than static.

Property	Values / Examples	Description
z-index	auto	Default value; stack based on order in HTML.
	<integer> (e.g. 10, 999)	Elements with higher z-index are in front.
	initial, inherit	Sets to default or parent's value.

Use Case Scenario: You want a modal popup to appear above a backdrop and all other page content.

```
<!DOCTYPE html>
<html>
<head>
<style>
.modal {
  position: absolute;
  top: 100px;
  left: 100px;
  width: 200px;
  height: 150px;
  background-color: white;
  border: 2px solid #000;
  z-index: 10;
}

.backdrop {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0,0,0,0.5);
  z-index: 5;
}
```

```
</style>
</head>
<body>
<div class="backdrop"></div>
<div class="modal">This modal appears above the backdrop.</div>
</body>
</html>
```

24. CSS Overflow

The overflow property controls what happens when content overflows the bounds of its container.

Property	Values / Examples	Description
overflow	visible (default)	Content is not clipped; overflows the element.
	hidden	Content is clipped; rest is hidden.
	scroll	Adds scrollbars even if not needed.
	auto	Adds scrollbars only when necessary.
overflow-x	Same values	Controls horizontal overflow.
overflow-y	Same values	Controls vertical overflow.

Use Case: You want to create a scrollable box with limited height for a chat window or long content.

```
<!DOCTYPE html><html>
<head>
<style>
.scroll-box {
  width: 300px;
  height: 150px;
  border: 2px solid #ccc;
  overflow: auto;
  padding: 10px;
}
```

```
</style>
</head>
<body>
<div class="scroll-box">
  <p>This is a scrollable container. Add enough content here to trigger overflow behavior. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed luctus ultricies urna, a tincidunt risus facilisis in.</p>
</div>
</body></html>
```

25. CSS Float

The float property is used **to position elements to the left or right** of their container, allowing text or other content to wrap around them.

Property	Values / Examples	Description
float	left, right	Floats the element to that side.
	none (default)	No float applied.
clear	left, right, both	Stops following elements from wrapping.

Use Case: You want to align an image to the left and let text wrap around it.

```
<!DOCTYPE html><html><head>
<style>
img {
  float: left;
  margin-right: 15px;
  width: 150px;
}
.clear {clear: both;}
</style>
</head>
<body>
<img src="https://www.w3schools.com/w3images/avatar2.png" alt="Avatar">
<p>This text wraps around the floated image..</p>
<div class="clear"></div>
<p>Text below floated element.</p>
</body></html>
```

26. CSS Inline-block

The inline-block display value allows an element to behave like inline (no line break), while still supporting width, height, padding, and margin like a block.

Display Type	Description
inline	No new line; no control over size
block	Forces new line; full control over dimensions
inline-block	No new line + control over size (best of both worlds)

Use Case: You want to display multiple buttons or cards side-by-side with spacing and dimensions.

```
<!DOCTYPE html>
<html>
<head>
<style>
.card {
  display: inline-block;
  width: 200px;
  height: 100px;
  margin: 10px;
  background-color: #74b9ff;
  color: white;
  text-align: center;
  line-height: 100px;
  border-radius: 8px;
}
</style>
</head>
<body>
<div class="card">Card 1</div>
<div class="card">Card 2</div>
<div class="card">Card 3</div>
</body>
</html>
```

27. CSS Combinators

Combinators define relationships between selectors. They allow targeting elements based on their relationship with other elements in the DOM.

Combinator	Syntax / Example	Description
Descendant	div p	Selects <p> inside <div> (any level deep).
Child	div > p	Selects <p> that is a direct child of <div>.
Adjacent	div + p	Selects the first <p> immediately after <div>.
General Sibling	div ~ p	Selects all <p> after <div> (siblings).

Use Case Scenario: You want to style paragraphs only inside specific containers, or the first paragraph after a heading.

```
<!DOCTYPE html>
<html>
<head>
<style>
.container p {
  color: blue;
}
.container > p {
  font-weight: bold;
}
h2 + p {
  color: red;
}
.box ~ p {
  font-style: italic;
}
</style>
</head>
<body>
<div class="container">
  <p>Child paragraph</p>
  <div>
    <p>Nested paragraph</p>
```

```
  </div>
</div>
<h2>Heading</h2>
<p>Adjacent paragraph (styled red)</p>
<div class="box">Box Element</div>
<p>Sibling 1 (styled italic)</p>
<p>Sibling 2 (styled italic)</p>
</div>
</body>
</html>
```

28. CSS Dropdown (with Hover)

Dropdowns can be created using hover, positioning, and display properties in CSS without needing JavaScript.

Feature	Description
position: relative / absolute	Used to overlay dropdown on top of other content.
display: none + display: block	Show/hide dropdown content.
hover selector	Triggers the dropdown when mouse is over the parent element.

Use Case: You want a navigation bar item to show more options when hovered.

```
<!DOCTYPE html>
<html>
<head>
<style>
.dropdown {
  position: relative;
  display: inline-block;
}
.dropdown-content {
  display: none;
  position: absolute;
```

```
  background-color: #f1f1f1;
  min-width: 160px;
  box-shadow: 0 0 10px rgba(0,0,0,0.2);
  z-index: 1;
}
.dropdown-content a {
  color: black;
  padding: 12px 16px;
  display: block;
  text-decoration: none;
}
.dropdown-content a:hover {
  background-color: #ddd;
}
.dropdown:hover .dropdown-content {
  display: block;
}
</style>
</head>
<body>
<div class="dropdown">
  <button>Menu</button>
  <div class="dropdown-content">
    <a href="#">Link 1</a>
    <a href="#">Link 2</a>
    <a href="#">Link 3</a>
  </div>
</div>
</body>
</html>
```

29. CSS Pseudo-classes

A **pseudo-class** is used to define a special state of an element. It allows you to style an element when it is in a particular state — like when it's being hovered, focused, the first/last child, or when a checkbox is checked.

Common Pseudo-classes and Their Use:

Pseudo-class	Example	Description
:hover	a:hover	When the mouse is over the element.
:focus	input:focus	When the element is in focus (e.g. via tab or click).
:first-child	li:first-child	Targets the first child of its parent.
:last-child	p:last-child	Targets the last child.
:nth-child(n)	li:nth-child(2)	Targets the nth element in a group.
:checked	input:checked	Styles a checkbox or radio that is checked.
:disabled	input:disabled	Targets form elements that are disabled.
:not(selector)	p:not(.intro)	Selects everything except the specified element.

Use Case Scenario: You want to:

- Highlight every odd row in a table,
- Change input border when focused,

Practical:-

```
<!DOCTYPE html>
<html>
<head>
<style>
input:focus {
  border: 2px solid #00cec9;
}

li:first-child {
  font-weight: bold;
}

li:nth-child(odd) {
```

```
  background-color: #f1f2f6;
}

input:checked + label {
  color: green;
}
</style>
</head>
<body>
<h3>Focus and Child Selectors</h3>
<input type="text" placeholder="Click or Tab here">
<ul>
  <li>Item One</li>
  <li>Item Two</li>
  <li>Item Three</li>
</ul>
<h3>Checkbox Style</h3>
<input type="checkbox" id="opt1" checked>
<label for="opt1">Option 1</label>
<input type="checkbox" id="opt2">
<label for="opt2">Option 2</label>
</body>
</html>
```

30. CSS Pseudo-elements

Pseudo-elements allow you to **style specific parts of an element** (like the first letter or line of text), or insert content before or after an element using CSS only.

Pseudo-element	Example	Description
::before	p::before	Inserts content before the actual content.
::after	h1::after	Inserts content after the element.
::first-letter	p::first-letter	Targets the first letter of a block.
::first-line	p::first-line	Targets the first line of a paragraph.
::selection	::selection	Styles text selected by user.

Use Case Scenario:

- Add icons using only CSS,
- Stylize the first letter of an article like a drop cap.

Practical:-

```
<!DOCTYPE html>
<html>
<head>
<style>
p::first-letter {
  font-size: 200%;
  color: #d63031;
  font-weight: bold;
}

p::first-line {
  font-style: italic;
}

h1::after {
  content: " ✨ ";
  color: gold;
}

::selection {
  background: #81ecec;
  color: black;
}
</style>
</head>
<body>
<h1>Article Title</h1>
<p>This paragraph demonstrates pseudo-elements. The first letter is stylized, and the first line appears italic. Try selecting the text too!</p></body>
</html>
```

31. CSS RWD (Responsive Web Design)

Responsive Web Design (RWD) is a design approach that ensures a website looks good on all devices — desktops, tablets, and smartphones — by using flexible layouts, flexible images, and media queries.

Method	Description
Relative units	Use %, vw, vh, em instead of fixed px for sizes.
Media queries	Apply different styles for different screen sizes.
Max-width	Prevents elements from becoming too large on big screens.
Flexbox/Grid	Layout systems that adapt naturally to different screen sizes.

Use Case: You want a layout that adjusts text size, margins, or column layout depending on whether it's viewed on mobile or desktop.

```
<!DOCTYPE html>
<html>
<head>
<style>
.container {
 width: 90%;
 max-width: 960px;
 margin: auto;
 background-color: #dff9fb;
 padding: 20px;
 font-family: Arial;
}
@media screen and (max-width: 600px) {
 .container {
  font-size: 14px;
  padding: 10px;
  background-color: #c8d6e5;
 }
}
</style>
</head>
<body>
```

```
<div class="container">
  <h2>Responsive Web Design</h2>
  <p>This layout adjusts based on the screen width. Resize your browser to see the
effect! </p>
</div>
</body>
</html>
```

32. CSS RWD Images

Responsive images adjust automatically to fit the container and screen size, preventing overflow or distortion.

Property	Values	Description
max-width	100%	Ensures image doesn’t overflow its container.
height	auto	Maintains aspect ratio.
width	100%	Optional — stretches image to container.

Use Case : You want images in articles to scale down gracefully on smaller screens.

```
<!DOCTYPE html>
<html>
<head>
<style>
img.responsive {
  max-width: 100%;
  height: auto;
  display: block;
  margin: auto;
}
</style>
</head>
<body>
<img class="responsive" src="https://www.w3schools.com/w3images/lights.jpg"
alt="Responsive Image">
</body>
</html>
```

33. CSS RWD Videos

Videos can be made responsive by embedding them inside a container that maintains aspect ratio using padding techniques.

CSS Trick	Description
padding-bottom	Creates aspect ratio (e.g., 56.25% for 16:9).
position: absolute	Makes video fit inside relative wrapper.
max-width: 100%	Prevents overflow and ensures responsiveness.

Use Case :-You want a YouTube video to resize on phones and desktops automatically.

```
<!DOCTYPE html><html>
<head><style>
.video-container {
  position: relative;
  padding-bottom: 56.25%; /* 16:9 aspect ratio */
  height: 0;
  overflow: hidden;
  max-width: 100%;
}
.video-container iframe {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
}
</style>
</head>
<body>
<div class="video-container">
  <iframe src="https://www.youtube.com/embed/tgbNymZ7vqY"
frameborder="0" allowfullscreen></iframe>
</div>
</body></html>
```

34. CSS Grid

CSS Grid is a two-dimensional layout system that enables designers to arrange elements in **rows and columns**. It's ideal for creating **complex layouts** that are responsive and scalable.

Property	**Values / Examples**	**Description**
display	grid	Turns a container into a grid layout.
grid-template-columns	1fr 1fr, repeat(3, 1fr)	Defines the number and width of columns.
grid-template-rows	100px auto, repeat(2, 1fr)	Defines the number and height of rows.
grid-column-gap / column-gap	20px	Space between columns.
grid-row-gap / row-gap	20px	Space between rows.
gap	20px or 20px 10px	Shorthand for both row and column gaps.
grid-column / grid-row	1 / 3, 2 / span 2	Specifies item placement across grid lines.

Use Case: You want to create a responsive three-column card layout that auto-wraps on smaller devices.

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 15px;
  padding: 20px;
}
.grid-item {
  background-color: #0984e3;
```

```
  color: white;
  padding: 30px;
  text-align: center;
  border-radius: 8px;
  font-weight: bold;
}
</style>
</head>
<body>
<div class="grid-container">
  <div class="grid-item">Item 1</div>
  <div class="grid-item">Item 2</div>
  <div class="grid-item">Item 3</div>
  <div class="grid-item">Item 4</div>
</div>
</body>
</html>
```

35. CSS Navigation Bar

A navigation bar (navbar) is a collection of links displayed either horizontally or vertically, often styled using flex or inline-block with hover effects.

Property	Values / Examples	Description
display	flex, inline-block	Controls layout style of links.
text-decoration	none	Removes underline from links.
hover	a:hover	Styles links on mouse hover.
background-color	Any color value	Background of nav or links.
padding, margin	10px 20px	Creates spacing inside links.

Use Case : You want a horizontal top menu with styled links and hover effects.

Practical:-

```
<!DOCTYPE html>
<html>
<head>
<style>
.navbar {
  background-color: #2d3436;
  overflow: hidden;
  display: flex;
  justify-content: center;
}

.navbar a {
  color: white;
  padding: 14px 20px;
  text-decoration: none;
  display: block;
}

.navbar a:hover {
  background-color: #636e72;
}

</style>
</head>
<body>

<div class="navbar">
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Projects</a>
  <a href="#">Contact</a>
</div>

</body>
</html>
```

36. CSS Rounded Corners

Rounded corners are created using the border-radius property. It allows partial or full corner rounding on all or specific sides.

Property	Values / Examples	Description
border-radius	10px, 50%, 10px 20px	Rounds all corners or specific ones.
border-top-left-radius	10px	Rounds only the top-left corner.
border-top-right-radius	10px	Rounds only the top-right corner.
border-bottom-left-radius	10px	Rounds only the bottom-left corner.
border-bottom-right-radius	10px	Rounds only the bottom-right corner.

Use Case: You want buttons and cards to have a modern look with smooth corners.

```
<!DOCTYPE html>
<html>
<head>
<style>
.round-box {
  width: 250px;
  padding: 20px;
  background-color: #ffeaa7;
  border-radius: 15px;
  margin: 20px auto;
  text-align: center;
}
</style>
</head>
<body>
<div class="round-box">
  I have rounded corners!
</div>
</body>
</html>
```

37. CSS Border Images

The border-image property allows you to use an image as the border around an element. It's a powerful way to style borders with custom graphics.

Property	Values / Examples	Description
border-image-source	url('border.png')	Sets the image for the border.
border-image-slice	30 or 30 10	Determines how to slice the image (inward).
border-image-width	auto, 10%, 15px	Sets the width of the border image.
border-image-repeat	stretch, repeat, round, space	Determines how the image is repeated.
border-image	Shorthand: source slice / width / repeat	Combines all into one property.

Use Case: You want to style an info box with a custom graphic frame instead of standard borders.

Practical:-

```
<!DOCTYPE html><html><head>
<style>
.border-img-box {
  border: 10px solid transparent;
  padding: 20px;
  border-image: url('https://www.w3schools.com/css/border.png') 30 round;
}
</style>
</head>
<body>
<div class="border-img-box">
  This box uses a border image!
</div>
</body>
</html>
```

38. CSS Multiple Backgrounds

CSS allows you to apply **more than one background image** to an element by comma-separating each background-* value. You can layer multiple images with individual position, size, and repeat behavior.

Property	Example	Description
background-image	url(img1.jpg), url(img2.png)	Sets multiple background images. The first is on top.
background-position	center center, left top	Sets position for each image.
background-size	cover, contain	Sets individual size for each image.
background-repeat	no-repeat, repeat-x	Controls how each background image is repeated.
background-attachment	fixed, scroll	Controls scroll behavior for each layer.
background (shorthand)	Combines all the above	Efficiently defines multiple backgrounds with full control.

Use Case : You want to apply a pattern on top of a background image without needing extra elements.

Practical Example:

```
<!DOCTYPE html><html>
<head>
<style>
.multi-bg {
  height: 300px;
  width: 100%;
  background-image: url('https://www.w3schools.com/css/img_flwr.gif'),
                url('https://www.w3schools.com/css/paper.gif');
  background-position: center, left top;
  background-repeat: no-repeat, repeat;
  background-size: 100px 100px, auto;
  border: 2px solid #ccc;
  padding: 20px;
}
```

```
</style>
</head>
<body>
<div class="multi-bg">
  This div has a flower icon layered over a paper pattern.
</div>
</body>
</html>
```

39. CSS Color & Color Keywords

CSS provides various ways to define colors — including predefined keywords, RGB, HEX, HSL, and transparent values.

Method	Syntax / Example	Description
Color name	color: red;	Uses predefined keywords like red, blue, green, etc.
HEX code	color: #ff0000;	Uses hexadecimal values (#RRGGBB).
RGB	color: rgb(255, 0, 0);	Uses Red, Green, Blue values.
RGBA	color: rgba(255, 0, 0, 0.5);	Adds opacity to RGB (0.0 transparent to 1.0).
HSL	color: hsl(0, 100%, 50%);	Hue (0-360), Saturation, Lightness format.
HSLA	color: hsla(0, 100%, 50%, 0.3);	Same as HSL, with added alpha (opacity).
transparent	background-color: transparent;	Fully see-through element.

Use Case Scenario: You want to apply background transparency on hover and use brand-specific HEX or RGB color values for consistency.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.color-demo {
  padding: 20px;
  margin: 10px;
```

```
  color: white;
  font-weight: bold;
}

.named {
  background-color: navy;
}

.hex {
  background-color: #e84393;
}

.rgb {
  background-color: rgb(52, 152, 219);
}

.rgba {
  background-color: rgba(52, 152, 219, 0.5);
  color: black;
}

.hsl {
  background-color: hsl(120, 60%, 40%);
}

.hsla {
  background-color: hsla(120, 60%, 40%, 0.3);
  color: black;
}
</style>
</head>
<body>

<div class="color-demo named">Named Color (navy)</div>
<div class="color-demo hex">HEX Color (#e84393)</div>
<div class="color-demo rgb">RGB Color</div>
<div class="color-demo rgba">RGBA Color with Opacity</div>
<div class="color-demo hsl">HSL Color</div>
<div class="color-demo hsla">HSLA Transparent</div>

</body>
</html>
```

40. CSS Gradients

Gradients create smooth transitions between colors without needing actual images. CSS supports linear, radial, and conic gradients.

Gradient Type	Syntax Example	Description
Linear Gradient	linear-gradient(to right, red, yellow)	Gradient that moves in a straight line (directional).
Radial Gradient	radial-gradient(circle, red, yellow)	Gradient radiating from center or a point outward.
Conic Gradient	conic-gradient(from 90deg, red, yellow, blue) (Modern browsers)	Gradient that sweeps around a center point in a circle.
Repeating Gradient	repeating-linear-gradient(45deg, red 0px, yellow 10px)	Repeats the gradient pattern infinitely.

Use Case: You want a button with a vertical gradient or a background with a circular center-out fade.

Practical Example:

```
<!DOCTYPE html>

<html>
<head>
<style>
.gradient-box
{
  padding: 20px;
  color: white;
  margin-bottom: 20px;
  text-align: center;
  font-weight: bold;
  border-radius: 8px;
}

.linear
{
  background: linear-gradient(to right, #6a11cb, #2575fc);
```

```
}

.radial
 {
  background: radial-gradient(circle, #ff9ff3, #f368e0);
}

.conic
{
  background: conic-gradient(from 0deg, red, yellow, lime, cyan, blue, magenta,
red);
}

.repeating
 {
  background: repeating-linear-gradient(
    45deg,
    #ff6b6b,
    #ff6b6b 10px,
    #feca57 10px,
    #feca57 20px
  );
}
</style>
</head>
<body>

<div class="gradient-box linear">Linear Gradient</div>
<div class="gradient-box radial">Radial Gradient</div>
<div class="gradient-box conic">Conic Gradient</div>
<div class="gradient-box repeating">Repeating Linear Gradient</div>

</body>
</html>
```

41. CSS Shadows

CSS shadows allow you to apply visual depth and soft focus to elements using **text shadows and box shadows**. These effects enhance UI elements like cards, buttons, and text.

Property	Example	Description
text-shadow	2px 2px 5px gray	Applies shadow to text. Format: x-offset y-offset blur color
box-shadow	4px 4px 10px rgba(0, 0, 0, 0.3)	Applies shadow to the box element.
	inset 2px 2px 5px #999	inset makes shadow appear inside the element.
	0 0 10px red, 0 0 20px orange	Multiple shadows can be applied using comma separation.

Use Case : You want to give a card a soft drop shadow and make text glow or appear embossed.

Practical Example:

```
<!DOCTYPE html><html><head>
<style>
.shadow-box {
  width: 300px;
  margin: 20px auto;
  padding: 20px;
  background-color: #fff;
  border: 1px solid #ddd;
  box-shadow: 4px 4px 15px rgba(0,0,0,0.2);
  text-align: center;
}

.shadow-text {
  font-size: 24px;
  color: #2d3436;
  text-shadow: 2px 2px 4px #b2bec3;
}
```

```
.inset-box {
  width: 300px;
  padding: 20px;
  margin: 20px auto;
  background-color: #dfe6e9;
  box-shadow: inset 5px 5px 10px #b2bec3;
}
</style></head>
<body>
<div class="shadow-box">
  <p class="shadow-text">Text with Shadow</p>
</div>
<div class="inset-box">
  Inset Box Shadow Example
</div>
</body></html>
```

42. CSS Text-effects

CSS text effects enhance text appearance and readability. These include transformations, decorations, spacing, overflow handling, and more.

Property	Values / Examples	Description
text-transform	uppercase, lowercase, capitalize	Alters case of text.
text-decoration	underline, line-through, overline, none	Adds or removes text lines.
text-align	left, center, right, justify	Aligns text within a block.
letter-spacing	2px, -1px	Adjusts spacing between characters.
word-spacing	4px, -2px	Adjusts spacing between words.
line-height	1.5, 2em	Sets vertical spacing between lines.
text-overflow	ellipsis	Truncates overflowing text with ....
white-space	nowrap, pre, normal	Controls how text wraps or preserves space.

Property	Values / Examples	Description
overflow	hidden, scroll	Needed for truncation effects.

Use Case: You want headings in uppercase, long content to end with ellipsis, and stylized links with underline and spacing.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.text-transform {
  text-transform: uppercase;
  font-weight: bold;
  color: #6c5ce7;
}

.text-decoration {
  text-decoration: underline;
  color: #00cec9;
  letter-spacing: 2px;
}

.truncate {
  width: 200px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
  border: 1px solid #ccc;
  padding: 5px;
}
</style>
</head>
<body>
<h2 class="text-transform">Transformed Heading</h2>
<p class="text-decoration">This is an underlined link-style text</p>
<div class="truncate">
  This is a long sentence that will be truncated with ellipsis.
</div>
</body></html>
```

43. CSS 2D Transforms

2D transforms allow elements to be moved, rotated, scaled, and skewed in two-dimensional space without affecting surrounding elements.

Common 2D Transform Functions:

Function	Example	Description
translate(x, y)	transform: translate(50px, 20px)	Moves the element along the X and Y axes.
rotate(deg)	transform: rotate(45deg)	Rotates the element clockwise (negative = counterclockwise).
scale(x, y)	transform: scale(1.5, 2)	Scales width (X) and height (Y).
skew(x, y)	transform: skew(20deg, 10deg)	Skews the element along the X and

43.1 translate(x, y)

Moves an element from its current position using x (horizontal) and y (vertical) offsets.

```
<!DOCTYPE html>
<html>
<head>
<style>

.translate-box {
  width: 100px;
  height: 100px;
  background: #74b9ff;
  transform: translate(50px, 20px);
}
</style>
</head>
<body>
<div class="translate-box">Translate</div>
</body>
</html>
```

43.2 rotate(angle)

Rotates the element clockwise by the given angle. Negative values rotate counter-clockwise.

```
<!DOCTYPE html>
<html>
<head>
<style>
.rotate-box {
  width: 100px;
  height: 100px;
  background: #fab1a0;
  transform: rotate(45deg);
}
</style>
</head>
<body>
<div class="rotate-box">Rotate</div>
</body>
</html>
```

43.3 scale(x, y):- Increases or decreases the size of the element. Values greater than 1 enlarge, less than 1 shrink.

```
<!DOCTYPE html>
<html>
<head>
<style>
.scale-box {
  width: 100px;
  height: 100px;
  background: #55efc4;
  transform: scale(1.5, 1.2); /* 150% width, 120% height */
}
</style>
</head>
<body>
<div class="scale-box">Scale</div>
</body>
</html>
```

43. 4 skew(x-angle, y-angle) :- Tilts the element along the X and/or Y axes.

```
<!DOCTYPE html>
<html>
<head>
<style>
.skew-box {
  width: 120px;
  height: 80px;
  background: #ffeaa7;
  transform: skew(20deg, 10deg);
}
</style>
</head>
<body>
<div class="skew-box">Skew</div>
</body>
</html>
```

43.5 matrix(a, b, c, d, e, f)

A combination of all transforms in a single function.

- a = scaleX, b = skewY
- c = skewX, d = scaleY
- e = translateX, f = translateY

```
<!DOCTYPE html><html><head>
<style>
.matrix-box {
  width: 120px;
  height: 80px;
  background: #e17055;
  color: white;
  transform: matrix(1, 0.5, -0.5, 1, 30, 20);
}
</style></head>
<body>
<div class="matrix-box">Matrix</div>
</body></html>
```

44. CSS 3D Transforms

CSS 3D transforms add **depth** to web elements, allowing them to be rotated or transformed in three-dimensional space using the transform property along the X, Y, and Z axes.

To make 3D transforms visible, perspective must be applied to the parent or the element.

Common 3D Transform Functions:

Function	Syntax Example	Description
rotateX(angle)	transform: rotateX(45deg)	Rotates the element around the X-axis.
rotateY(angle)	transform: rotateY(45deg)	Rotates the element around the Y-axis.
rotateZ(angle)	transform: rotateZ(45deg)	Rotates the element around the Z-axis (same as 2D rotate).
translateZ(z)	transform: translateZ(100px)	Moves the element toward or away from the viewer.
scaleZ(z)	transform: scaleZ(1.5)	Scales the element along the Z-axis.
perspective(n)	perspective: 800px	Defines 3D perspective (should be on parent).
transform-style	preserve-3d	Allows nested elements to remain in 3D space.

Use Case: You want a card to flip or rotate in 3D space to create modern UI effects like a flip animation.

Practical Example: Basic 3D Rotation

```
<!DOCTYPE html>
<html>
<head>
<style>
.scene {
  width: 200px;
  height: 200px;
  perspective: 600px;
  margin: 50px auto;
}
```

```
.cube {
  width: 100%;
  height: 100%;
  background-color: #74b9ff;
  transform: rotateX(45deg) rotateY(45deg);
  color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: bold;
}
</style></head>
<body>
<div class="scene">
  <div class="cube">3D Rotate</div>
</div></body></html>
```

Note: To animate 3D transforms, combine with transition or keyframes.

- Use transform-style: preserve-3d; to allow child elements to maintain depth in nested transforms.

45. CSS Transitions

CSS Transitions allow you to **smoothly change property values** over time. They are ideal for hover effects, color shifts, movements, and more.

Property	Example	Description
transition-property	background-color, all	Specifies which property to animate.
transition-duration	0.5s, 1s	Time it takes to complete transition.
transition-timing-function	ease, linear, ease-in-out	Defines speed curve.
transition-delay	0.2s	Wait before transition starts.
transition	Shorthand	property duration timing delay

Use Case: You want a button to change color smoothly on hover and move slightly when clicked.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.button
 {
  background-color: #00cec9;
  color: white;
  padding: 12px 24px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  transition: background-color 0.3s ease, transform 0.2s ease;
}

.button:hover
{
  background-color: #0984e3;
}

.button:active
{
  transform: scale(0.95);
}

</style>
</head>

<body>
<button class="button">Hover or Click Me</button>
</body>
</html>
```

46. CSS Animations

CSS animations let you animate transitions between CSS property values over time using @keyframes.

Property	Example	Description
@keyframes	@keyframes slide { ... }	Defines animation steps.
animation-name	slide	Binds the animation to the element.
animation-duration	2s, 1s	Duration of the animation.
animation-delay	0.5s	Delay before animation starts.
animation-iteration-count	infinite, 1, 3	Number of times animation plays.
animation-direction	normal, reverse, alternate	Direction of animation.
animation-timing-function	ease, linear, ease-in-out	Speed curve.
animation-fill-mode	forwards, backwards, both	How element behaves before/after animation.

Use Case: You want to slide in a banner, pulse a button, or rotate an icon continuously.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
@keyframes slideIn {
 from {
  transform: translateX(-100%);
  opacity: 0;
 }
 to {
  transform: translateX(0);
  opacity: 1;
 }
}
.animated-box {
 width: 200px;
```

```
 height: 100px;
 background-color: #81ecec;
 animation-name: slideIn;
 animation-duration: 1s;
 animation-fill-mode: forwards;
}
</style>
</head>
<body>
<div class="animated-box">Animated In</div>
</body>
</html>
```

47. CSS Tooltips

Tooltips are small info boxes that appear when the user hovers over an element, created using the ::after or ::before pseudo-elements with content.

Concept	**Description**
position: relative	Needed on parent.
::after / ::before	Used to insert tooltip text.
visibility, opacity	Used to show/hide tooltip.
transition	Adds smooth fade effect.

Use Case: You want to show a short label when users hover over a button or icon.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.tooltip {
 position: relative;
 display: inline-block;
 cursor: pointer;
}
```

```
.tooltip::after {
  content: "Helpful Tooltip!";
  position: absolute;
  bottom: 125%;
  left: 50%;
  transform: translateX(-50%);
  background-color: #2d3436;
  color: #fff;
  padding: 6px 10px;
  border-radius: 4px;
  white-space: nowrap;
  opacity: 0;
  visibility: hidden;
  transition: opacity 0.3s;
}
.tooltip:hover::after {
  opacity: 1;
  visibility: visible;
}
</style>
</head>
<body>
<div class="tooltip">Hover me</div>
</body>
</html>
```

48. CSS Style Images

CSS can style images with borders, rounded corners, shadows, filters, and even hover effects — all without editing the image file.

Property	Example	Description
border	2px solid #000	Adds a border around the image.
border-radius	10px, 50%	Rounds corners or creates circular images.
box-shadow	0 4px 8px rgba(0,0,0,0.3)	Adds a drop shadow to image.
filter	grayscale(100%), blur(2px)	Applies image effects like blur, grayscale, etc.

Use Case: You want to create a circular profile image with shadow and grayscale on hover.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.styled-img {
  width: 150px;
  height: 150px;
  border-radius: 50%;
  border: 4px solid #00cec9;
  box-shadow: 0 4px 12px rgba(0,0,0,0.2);
  transition: filter 0.3s;
}

.styled-img:hover {
  filter: grayscale(100%);
}
</style>
</head>
<body>
<img class="styled-img" src="https://www.w3schools.com/howto/img_avatar.png"
alt="Profile">
</body>
</html>
```

49. CSS Image Reflection

Image reflection creates a **mirror-like effect** beneath an image. It's supported in **WebKit-based browsers** using the proprietary -webkit-box-reflect property.

Key Syntax:

```
-webkit-box-reflect: direction offset gradient;
```

- direction: below, above, left, right
- offset: how far the reflection is placed from the original
- gradient: fade-out effect (optional)

Use Case Scenario:

You want a product image to reflect below for a glossy or elegant effect.

Practical Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
.reflect-img {
  width: 200px;
  -webkit-box-reflect: below 5px linear-gradient(to bottom, transparent,
rgba(0,0,0,0.2));
}
</style>
</head>
<body>
<img class="reflect-img" src="https://www.w3schools.com/css/img_lights.jpg"
alt="Reflecting Image">
</body>
</html>
```

Note: -webkit-box-reflect only works in WebKit browsers (e.g., Chrome, Safari).